Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate linear convolution and circular convolution on discrete time signals.**	
Experiment No: 2	**Date:**	**Enrollment No: 92400133055**

Theory:
Linear convolution and circular convolution are mathematical operations used to combine two signals to obtain a third signal. They are widely used in various applications, such as signal processing, image processing, and audio processing.

Linear convolution calculates the sum of element-wise products of two signals, considering the full range of valid indices. It is typically used for finite-length signals and can produce an output signal that is longer than the input signals.

Circular convolution, on the other hand, calculates the sum of element-wise products of two signals, considering a periodic extension of the input signals. It is commonly used for periodic or infinite-length signals and produces an output signal with the same length as the input signals.

Flowchart:
The flowchart for the program will consist of the following steps:

Define two discrete-time signals (signal 1 and signal 2).
Compute the linear convolution of the two signals using numpy.convolve.
Compute the circular convolution of the two signals using numpy.fft.ifft.
Plot and display the linear and circular convolution results.
Save the linear and circular convolution results (optional).
Now, let's see the Python program that simulates linear convolution and circular convolution on discrete-time signal

```
import numpy as np
import matplotlib.pyplot as plt

def linear_convolution(signal1, signal2):
    # Compute the linear convolution
    linear_conv = np.convolve(signal1, signal2, mode='full')
    return linear_conv

def circular_convolution(signal1, signal2):
    # Compute the circular convolution
      # Compute the circular convolution
          If( len(signal1) > len(signal2) ):
          fft_length=len(signal1);
          else:
          fft_length=len(signal2);
    fft_signal1 = np.fft.fft(signal1, fft_length)
    fft_signal2 = np.fft.fft(signal2, fft_length)
    circular_conv = np.fft.ifft(fft_signal1 * fft_signal2)
    return circular_conv
```

https://github.com/ruhaanpathan/DSIP.git

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate linear convolution and circular convolution on discrete time signals.**	
Experiment No: 2	**Date:**	**Enrollment No: 92400133055**

```
# Define the discrete-time signals
signal1 = np.array([1, 2, 3, 4, 5])
signal2 = np.array([2, 4, 6, 8, 10])

# Compute the linear convolution
linear_conv = linear_convolution(signal1, signal2)

# Compute the circular convolution
circular_conv = circular_convolution(signal1, signal2)

# Plot the linear and circular convolution results
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.stem(linear_conv)
plt.title('Linear Convolution')
plt.xlabel('Sample')
plt.ylabel('Amplitude')

plt.subplot(2, 1, 2)
plt.stem(circular_conv)
plt.title('Circular Convolution')
plt.xlabel('Sample')
plt.ylabel('Amplitude')

plt.tight_layout()
plt.show()

# Save the linear and circular convolution results (optional)
# np.savetxt('linear_convolution.txt', linear_conv, delimiter=',')
# np.savetxt('circular_convolution.txt', circular_conv, delimiter=',')
```

Explanation:

We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.
The linear_convolution function computes the linear convolution between two signals using numpy.convolve with the mode set to 'full'.
The circular_convolution function computes the circular convolution between two signals using numpy.fft.fft and numpy.fft.ifft. It first calculates the FFT of both signals with a specified length and then computes the inverse FFT of the element-wise product.
In the main part of the program, we define two discrete-time signals (signal1 and signal2).
We compute the linear convolution by calling the linear_convolution function with signal1 and signal2.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate linear convolution and circular convolution on discrete time signals.**	
Experiment No: 2	**Date:**	**Enrollment No: 92400133055**

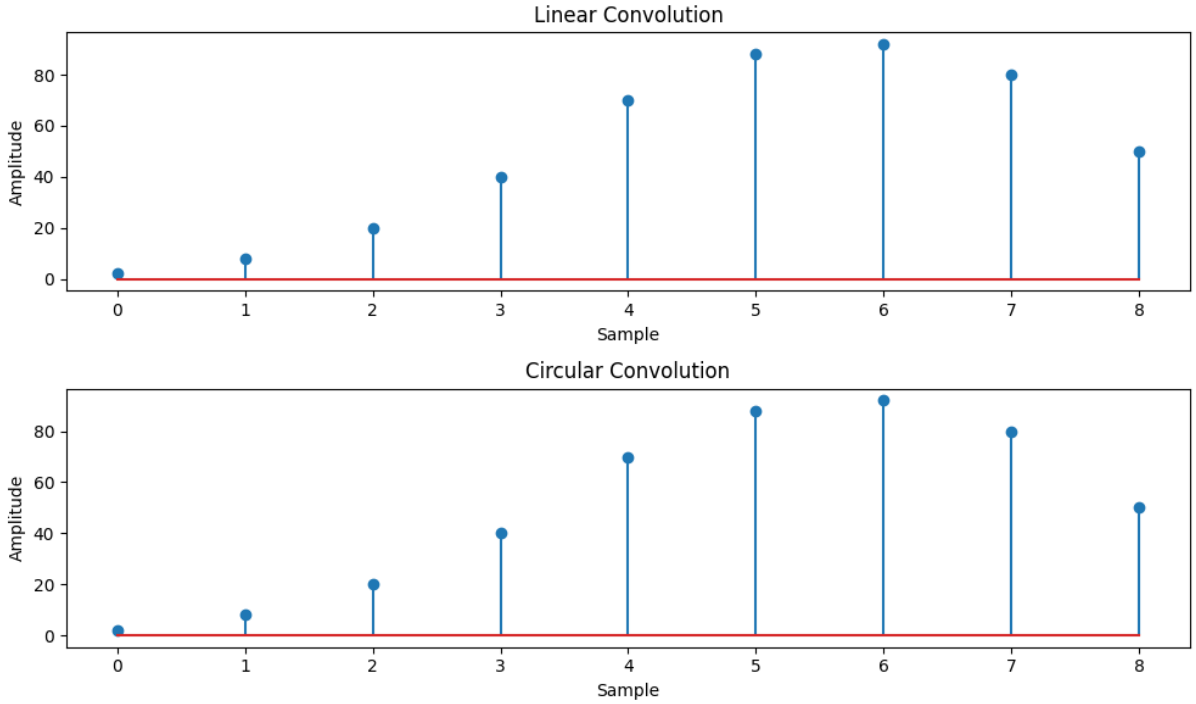
We compute the circular convolution by calling the circular_convolution function with signal1 and signal2.
We plot and display the linear and circular convolution results using matplotlib.pyplot.stem.
The linear and circular convolution results can be optionally saved to files using numpy.savetxt.
You can modify the signal1 and signal2 variables to use different discrete-time signals for convolution.

Uncomment the np.savetxt lines if you want to save the linear and circular convolution results to text files.

Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

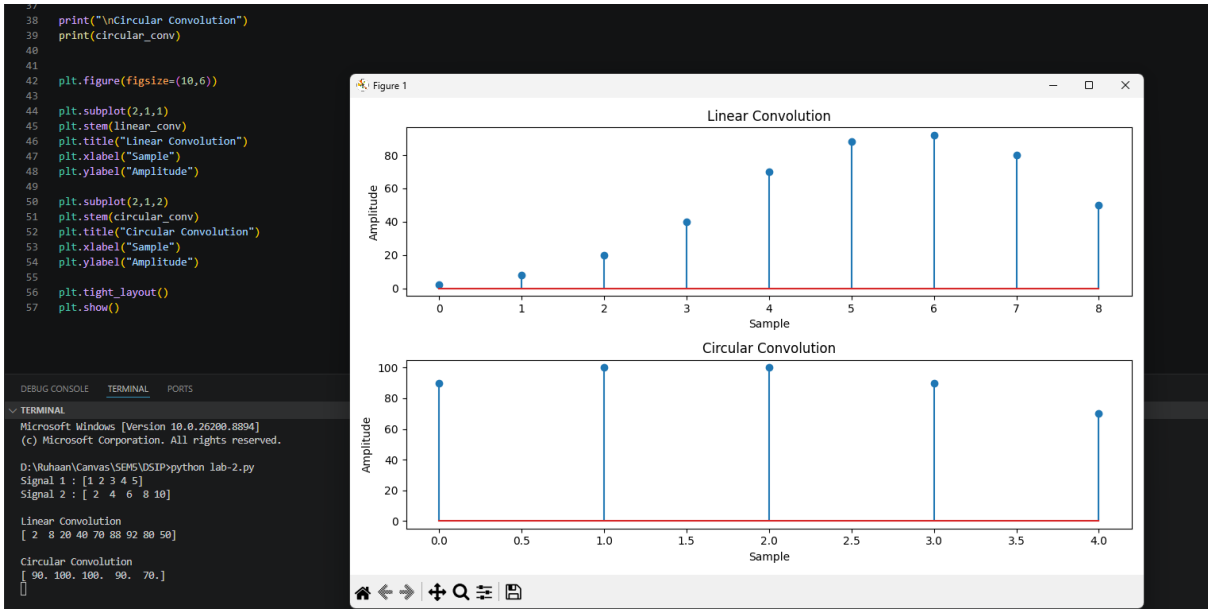
Expected Output:

A plot will be displayed showing the linear and circular convolution results.
The linear and circular convolution results will be saved to text files (optional).
Feel free to experiment with different discrete-time signals and observe the linear and circular convolution outputs.



https://github.com/ruhaanpathan/DSIP.git

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate linear convolution and circular convolution on discrete time signals.**	
Experiment No: 2	**Date:**	**Enrollment No: 92400133055**




```python
import numpy as np
import matplotlib.pyplot as plt


def linear_convolution(signal1, signal2):
    return np.convolve(signal1, signal2, mode='full')


def circular_convolution(signal1, signal2):
    fft_length = max(len(signal1), len(signal2))

    fft_signal1 = np.fft.fft(signal1, fft_length)
    fft_signal2 = np.fft.fft(signal2, fft_length)

    circular_conv = np.fft.ifft(fft_signal1 * fft_signal2)

    return np.real(circular_conv)

signal1 = np.array([1, 2, 3, 4, 5])
signal2 = np.array([2, 4, 6, 8, 10])


linear_conv = linear_convolution(signal1, signal2)
```

https://github.com/ruhaanpathan/DSIP.git

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate linear convolution and circular convolution on discrete time signals.**	
Experiment No: 2	**Date:**	**Enrollment No: 92400133055**

```
circular_conv = circular_convolution(signal1, signal2)


print("Signal 1 :", signal1)
print("Signal 2 :", signal2)

print("\nLinear Convolution")
print(linear_conv)

print("\nCircular Convolution")
print(circular_conv)


plt.figure(figsize=(10,6))

plt.subplot(2,1,1)
plt.stem(linear_conv)
plt.title("Linear Convolution")
plt.xlabel("Sample")
plt.ylabel("Amplitude")

plt.subplot(2,1,2)
plt.stem(circular_conv)
plt.title("Circular Convolution")
plt.xlabel("Sample")
plt.ylabel("Amplitude")

plt.tight_layout()
plt.show()
```

Conclusion:

The results demonstrated that linear convolution produces a longer output sequence, while circular convolution assumes periodic signals and produces an output of fixed length.

https://github.com/ruhaanpathan/DSIP.git

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate linear convolution and circular convolution on discrete time signals.**	
Experiment No: 2	**Date:**	**Enrollment No: 92400133055**

https://github.com/ruhaanpathan/DSIP.git